

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в глубину»

Выполнили:
студент группы 24ВВВЗ
Алмакаев Владимир

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2025

Цель работы – ознакомиться с алгоритмом обхода графа в глубину и освоить его практическую реализацию.

Общие сведения.

Обход графа – одна из наиболее распространенных операций с графами. Задачей обхода является прохождение всех вершин в графе. Обходы применяются для поиска информации, хранящейся в узлах графа, нахождения связей между вершинами или группами вершин и т.д.

Одним из способов обхода графов является поиск в глубину. Идея такого обхода состоит в том, чтобы начав обход из какой-либо вершины всегда переходить по первой встречающейся в процессе обхода связи в следующую вершину, пока существует такая возможность. Как только в процессе обхода исчерпаются возможности прохода, необходимо вернуться на один шаг назад и найти следующий вариант продвижения. Таким образом, итерационно выполняя описанные операции, будут пройдены все доступные для прохождения вершины. Чтобы не заходить повторно в уже пройденные вершины, необходимо их пометить как пройденные.

Таким образом, можно предложить следующую рекурсивную реализацию алгоритма обхода в глубину.

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОГ

1.1. для всех i положим $NUM[i] = False$ пометим как "не посещенную";

1.2. **ПОКА** существует "новая" вершина v

1.3. ВЫПОЛНЯТЬ DFS (v).

Алгоритм DFS(v):

2.1. пометить v как "посещенную" $NUM[v] = \text{True}$;

2.2. вывести на экран v;

2.3. **ДЛЯ** i = 1 **ДО** size_G **ВЫПОЛНЯТЬ**

2.4. **ЕСЛИ** $G(v,i) = 1$ **И** $NUM[i] = \text{False}$

2.5. **ТО**

2.6. {

2.7. DFS(i);

2.8. }

Реализация состоит из подготовительной части, в которой все вершины помечаются как не помеченные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3). И непосредственно процедуры обхода, которая помечает текущую (т.е. ту, в которой на текущей итерации находится алгоритм) вершину как посещенную (п. 2.1). Затем выводит номер текущей вершины на экран (п.2.2) и в цикле просматривает v-ю строку матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.4), то для этой вершины вызывается процедура обхода (п.2.7).

Задание:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>

void DFS(int** G, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d", s);

    for (int i = 0; i < numG; i++) {
        if (G[s][i] == 1 && visited[i] == 0)
            DFS(G, numG, visited, i);
    }
}

int main() {

    int** G;
    int numG, current;
    int* visited;

    printf("Input number of vertission:");
    scanf("%d", & numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++)
        G[i] = (int*)malloc(numG * sizeof(int));

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = 0; j < numG; j++) {
            G[i][j] = G[i][j] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
}
```

```

    printf("Input the start vert:");
    scanf("%d", &current);

    printf("Path: ");

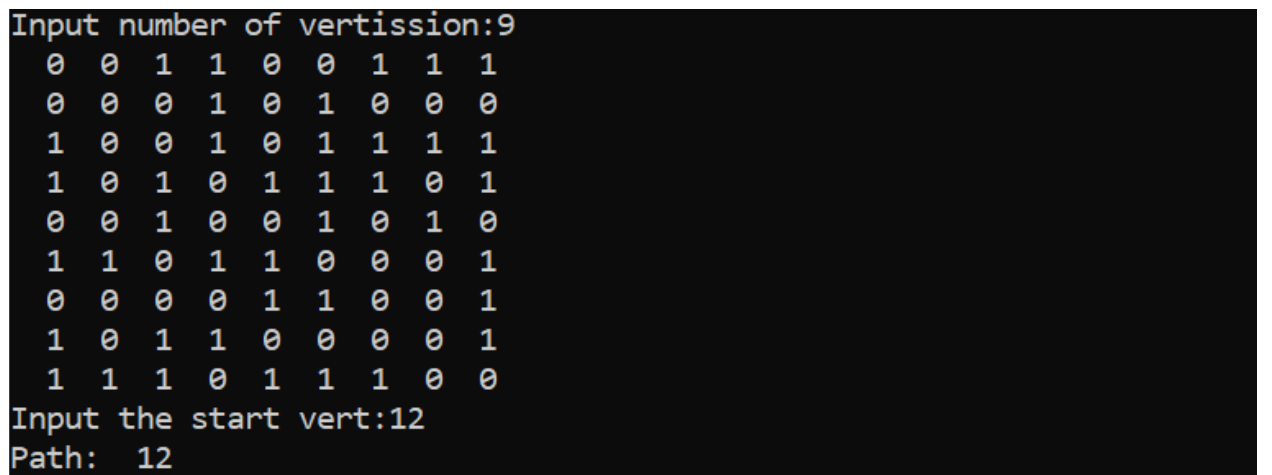
    DFS(G, numG, visited, current);

    free(visited);
    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);

    return 0;
}

```

Результат работы программы показан на рисунке 1



```

Input number of vertission:9
0 0 1 1 0 0 1 1 1
0 0 0 1 0 1 0 0 0
1 0 0 1 0 1 1 1 1
1 0 1 0 1 1 1 0 1
0 0 1 0 0 1 0 1 0
1 1 0 1 1 0 0 0 1
0 0 0 0 1 1 0 0 1
1 0 1 1 0 0 0 0 1
1 1 1 0 1 1 1 0 0
Input the start vert:12
Path: 12

```

Рисунок 1

Вывод: в ходе выполнения лабораторной работы ознакомился с алгоритмом обхода графа в глубину и освоил его практическую реализацию.